Assignment

BACS1024 Introduction to Computer Systems

Feb 2024 Semester

Student's name	:	(1) Wong Jin Xuan	(2) Dorcas Lim Yuan Yao	(3) Tan Yen Ping
Student's ID	:	(1) 2314630	(2) 2314535	(3) 2314615

Programme & Group : RDS Y1S2 G3

Tutor's name : _____

Date of Submission : 19/3/2024

Student	**Part I**	**Part II**				**Total / Grade**
	10	**10**	**10**	**10**	**10**	**50 marks**
(1) Wong Jin Xuan						
(2) Dorcas Lim Yuan Yao						
(3) Tan Yen Ping						

Comment:

Input three digit. Display three digit. Proper decimal value shows and calculated.
Able to retrieve system date and time.
Use of PROC.
Clear program comment.
Use of INT 10H for text color.
Clear formatted output.
Complex calculation involving discount, voucher and packaging fee if take away.
Involve the process of stock take and alert on stock having low re-order level.

Table of Contents

Coursework Declaration …….. 2

Assessment Rubrics ……. 3

System Synopsis (Proposal)

Proposed System's Name and Title …….. 4

Introduction and Background …… 4

Advantages and Justifications of Proposed System …….. 5

System Functions …….. 6

Mathematical Formula …….. 7

Flowchart ……..……. 9

User Manual with Screenshots …….……. 12

Advanced Features …….……..……. 24

Conclusion and Future Improvements…...……. 28

Coursework Declaration

Semester:	Feb 2024 Semester	**Course Code & Title:**	BACS1024 Introduction to Computer Systems

Declaration

I / we confirm that I /we have read and shall comply with all the terms and conditions of TAR UMT's plagiarism policy.

I / we declare that this submitted work is free from all form of plagiarism and for all intents and purposes is my own properly derived work.

I/We further confirm that the same work, where appropriate, has been verified by anti-plagiarism software_____*(please insert).*

Signature(s):	(1)	(2)	(.
Name(s):	(1) Wong Jin Xuan	(2) Dorcas Lim Yuan Yao	(3) Tan Yen Ping
Date:	18/3/2024		

Assessment Rubrics

Student's Name	**: (1) Wong Jin Xuan**	**(2) Dorcas Lim Yuan Yao**	**(3) Tan Yen Ping**
Student's ID	**: (1) 2314630**	**(2) 2314535**	**(3) 2314615**

Part I:
CLO3: Utilise relevant techniques, algorithms, mathematical and logic operations to solve programming problems. (C4, PLO6)

Criteria (Group basis)	**Incompetent**				**Highly competent**	**Remark**	**Score (10 marks)**
	0-2	**3-4**	**5-6**	**7-8**	**9-10**		
Use of information and digital technologies: Problem solving / communication / ethics (10 marks) ● Excellent application concept based on in-depth analysis. ● Well formatted report. ● Able to provide sound justification. ● Well-structured flow chart. ● Proper use of relevant techniques, algorithms, mathematical and logic operations							10

Part II:
CLO2: Produce practical solution related to computer systems by applying hardware, assembly language programming and operating system commands and scripting skills. (P3, PLO3)

Criteria (Group basis)	**Incompetent**				**Highly competent**	**Remark**	**Score (10 marks)**
	0-2	**3-4**	**5-6**	**7-8**	**9-10**		
Data structure / program comment / Program control (10 marks) ● Proper process flow. ● Proper naming convention used. ● Reusable program codes. ● Dynamic use of program code. ● Sufficient program comment.							10
I/O facilities (10 marks) ● Appropriate instructions. ● Appropriate I/O design. ● Sufficient I/O validation. ● Proper error message. ● User friendly.							10
Run / debug program (10 marks) ● No logical error. ● No run time error. ● Appropriate output. ● Appropriate output message. ● Appropriate program logic.							10
Advanced features (10 marks) ● Fulfill basic requirement. ● Use of complex algorithm. ● Able to solve complex problem. ● Include innovative idea. ● Well-designed program							10
							50 / 50

System Synopsis

Proposed System's Name / Title

Daily Fix Restaurant Point of Sales (POS) System

Introduction / Background

Daily Fix Restaurant was founded in 2024 which serves customers a variety of food and drinks. There is a wide range of diverse menu offered by the restaurant, such as breakfast, pasta, dessert, drinks, etc. It was located in Kuala Lumpur, Malaysia and attracts many local customers as well as tourists that visit there. Hence, it has an average of 60 to 100 customers visiting per day.

In order to make the ordering process of the restaurant more efficient, the Point of Sales (POS) system was introduced. POS is the point where the seller successfully makes an official sale transaction in exchange for goods or services with a customer and processes their payment. A reliable POS system allows a seller or profit organization in any industry to run their business more efficiently. The restaurant has a total of 8 workers with 3 of them in charge of taking orders and using the POS system. Their main task is to take orders from the customers, get the total sum of payment, make transaction payments, track the stock of ingredients and generate the sales report.

Advantages / Justifications of Proposed System

1. **Improves Productivity**
   By providing a fully reliable system that handles some tasks automatically, the staff can save time working on a single task, making their working process more efficient and effective. For example, workers can use the POS system proposed to calculate the total sum of payments (including all the tax, SST, discount and balance) for the customers instead of calculating it manually. It provides informed decision making for the staff, where they only have to choose the menu options available in the system such as what food or drinks being ordered, which payment method, dine-in or take away, and etc. This in turn makes the ordering process run as smoothly as possible according to the order sequence which reduces the worker's effort especially during peak hours.

2. **Reduce Human Error**
   There is a lower chance of making errors by using the POS system. This is because it is an automated system programmed to do mathematical calculations according to the algorithm designed beforehand. It reduces the chance of human error during manual calculation of the payment and transaction details. The system is designed to have validation of user input and will prompt error messages for reenter of input. This ensures the accuracy of the whole sales transaction process which then increases customers confidence and trust, leading to greater satisfaction of service provided to the customers.

3. **Improves Business Operations**
   - **Inventory Management**
     The system can monitor the ingredient usage by observing the quantity of certain food items sold in a day. It will notify the staff when certain food is sold out so that they would not take orders for food being sold out which may reduce customer satisfaction. The inventory tracking management will give a report of each item sold per day, which allows the restaurant manager to analyze the data in order to optimize ingredient preparation, reducing waste and cost. By tracking the inventory levels of the restaurant, it helps to prevent stockouts and ensures that the restaurant will always have sufficient ingredients.

   - **Sales Report Generating**
     The POS system proposed is designed to have the report generating function. It gives comprehensive and detailed information such as the quantity sold for each item, number of customers visited a day, total payment and profit generated. This data is useful for preparing ingredient stocks, production plans, observing popular food items, facilitating business decision-making, such as pricing strategies and managing production costs. It is a good way to boost the restaurant revenue by analyzing the data available in the summary report.

4. **Data Security**
   There is login and password functionality provided in the POS system to ensure that the user is the worker of the restaurant that handles the POS system. In addition, the POS system is fully usable by the Daily Fix restaurant only. Hence, all the data and reports are only visible by the staff and restaurant manager, ensuring data privacy protection. The tracking system of sales as well as inventory management also helps to prevent theft or fraud as the restaurant owner can make sure that the cash in the cash register / online transaction amount is equivalent to the sales report generated by the POS system.

System Functions

1. **Login**
   Login function is meant for staff to log in to the POS system using a unique ID and password. This function ensures that the authentication process is secure to avoid unauthorized access to the system or confidential information. Staff's account will be securely handled by the system, where only certain ID and correct password that match the account can logged into the POS system for further usage. In this way, the manager can also know which staff is using the system in a certain period of time.

2. **Register**
   Register function allows new users, usually restaurant staff, to create accounts in the POS system. During registration, staff details such as role of the staff are required. Staff are required to enter their desired password during registration under a new user ID in order to login into the POS system.

3. **Security**
   To keep user credentials safe from unauthorized use, the system applies secure password management techniques such as password masking (*) to reduce the risk of password leakage. To enhance security, it imposes password complexity constraints (such as minimum length). Based on the user's role, the system grants the proper access permissions. For example, only the supervisor or manager (admin) can view the sales report and allow registration of new users but the restaurant workers can only access other functions such as order preparation and stock tracking.

4. **Payment and Calculation**
   Employees can choose items from the menu to add to a customer's order using the POS system. The system also can handle modifications such as adding or canceling items. The POS system will calculate the total price, including taxes and any applicable deals. To fulfill different customer preferences, the POS system accepts various types of payment options, including cash, credit/debit cards, and mobile payments. Staff need to enter the amount paid by the customer if they are paying with cash so that the system can calculate the balance that needs to be returned.

5. **Stock Tracking**
   The stock tracking function enables real-time stock monitoring. Stock quantities are instantly updated as things are sold. In order to avoid stockouts, this function alerts the users when stock levels drop below indicated levels.

6. **Sales Report**
   Total revenue is computed by the POS system by adding up the sales of each food item made in a working day. The manager can see report data such as profit earned, number of customers, and quantity sold for each item. An analysis of profit breakdown by categories reveals which aspects of the restaurant's choices have the biggest impact on overall sales. Best sellers can also be determined through this function of report generation, which uses sales data analysis to determine the most favored menu items based on the (profit or) amount sold.

7. **Multi-level Menu Display and Selection**
   The POS system structures the system's functions into different categories such as making orders, stock balance, and sales reports. While making an order, there is another menu provided for the users to input the food categories (e.g. main dishes, desserts and drinks) to make it easier for staff to navigate and choose items during order entry. Users can also choose the type of payment through the menu selection in the payment section.

Mathematical Formula

- **Calculating Subtotal**
  a) Each item subtotal: itemSubtotal = quantity * unitPrice
     Code Variable:
     AX = USER_CHS[SI] * FOOD_PRICE[SI]

  b) Final summation of order: subTotal = (qty1 * unitPrice) + (qty2 * unitPrice) + …
     Code Variable:
     SUBTOTAL += AX

  Case Takeaway
  Constant packaging fee: RM0.20
  Final summation of order: subTotal += 0.20
  Code Variable: (20 → RM 0.20)
  SUBTOTAL = SUBTOTAL + 20 (treat decimal value as integer)

- **Service Tax**
  a) **SST (Government Tax 6%)**
     SSTTAX = A 6 % Service Tax which also called "taxable service" for customers (SST)
     Code Variable:
     HDR DB 100
     SIX DB 6
     SSTTAX = SUBTOTAL / HDR * SIX

  b) **Service Charge (10%)**
     SERVICETAX = A 10% Service charge for serving customers
     Code Variable:
     TEN DB 10
     SERVICETAX = SUBTOTAL / HDR * TEN

  Total After Tax
  SUBTOTAL += SSTTAX + SERVICE TAX

- **Discount / Voucher**
  a) **No discount**
     Initial set: DISCOUNT = 0

  b) **Member Discount (10%)**
     Only applicable for membership customer
     discount = subTotal * 0.10
     Code Variable:
     DISCOUNT = SUBTOTAL/ HDR * TEN

  c) **RM 7 Voucher (SAVE7)**
     Save7 = Voucher that can discount RM7 if purchase over RM25
     CMP SUBTOTAL,2500 (2500 → RM 25.00)
     discount += 7.00
     Code Variable:
     DISCOUNT += 700 (700 → RM7.00)

- **Total Payment Amount**
  SUBTOTAL = SUBTOTAL + SSTTAX + SERVICETAX - DISCOUNT
  (done while calculating and displaying the above)

- **Balance (cash payment)**
  CASHPAID = get input of amount of cash paid by customer
  BALANCE = CASHPAID - SUBTOTAL

- **Daily Sales Report**
  a) **Total Customer**
     CUSTOMERS = total customer in a day (initial 0)
     For every order: CUSTOMERS += 1

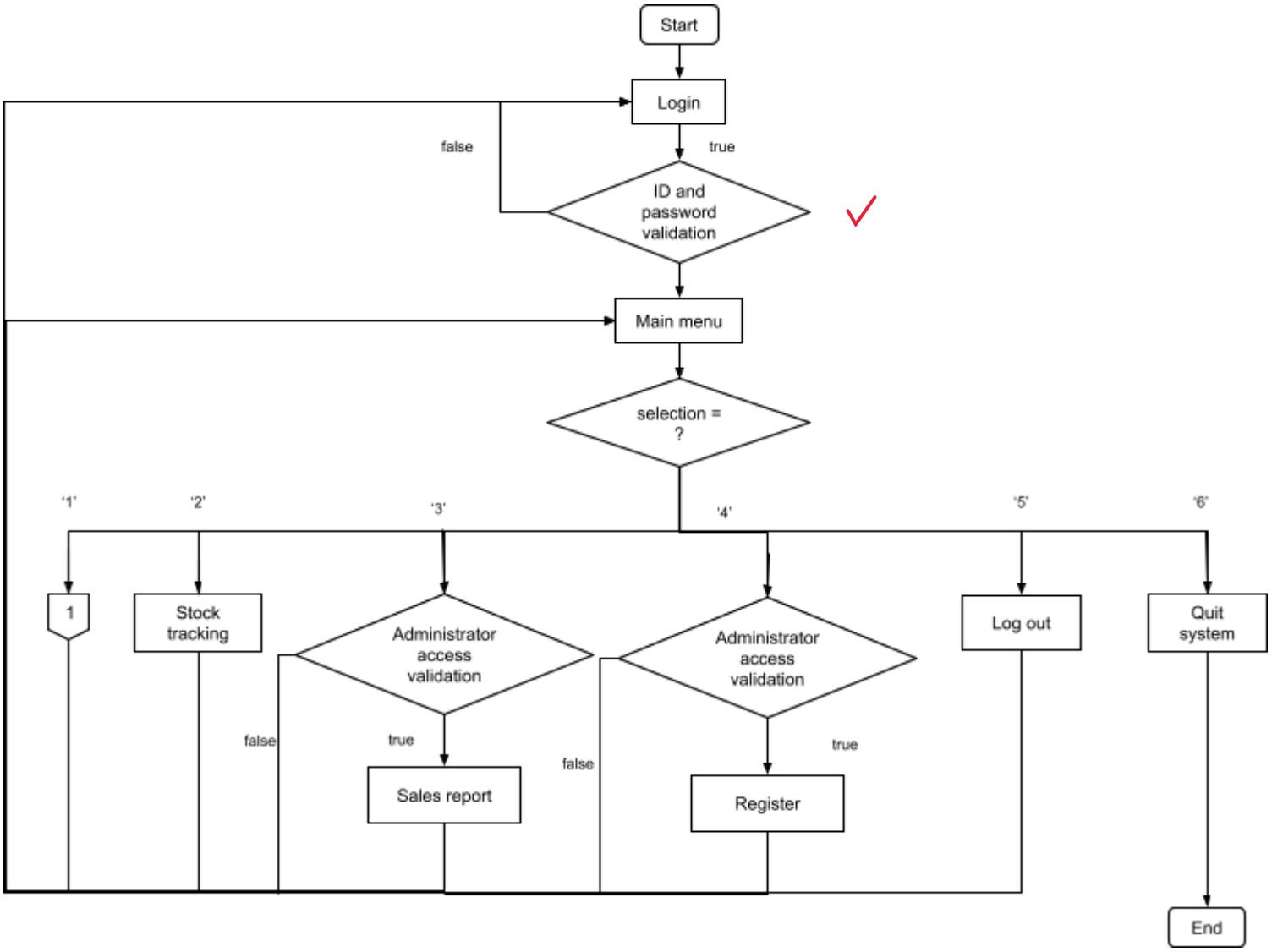
  b) **Table**

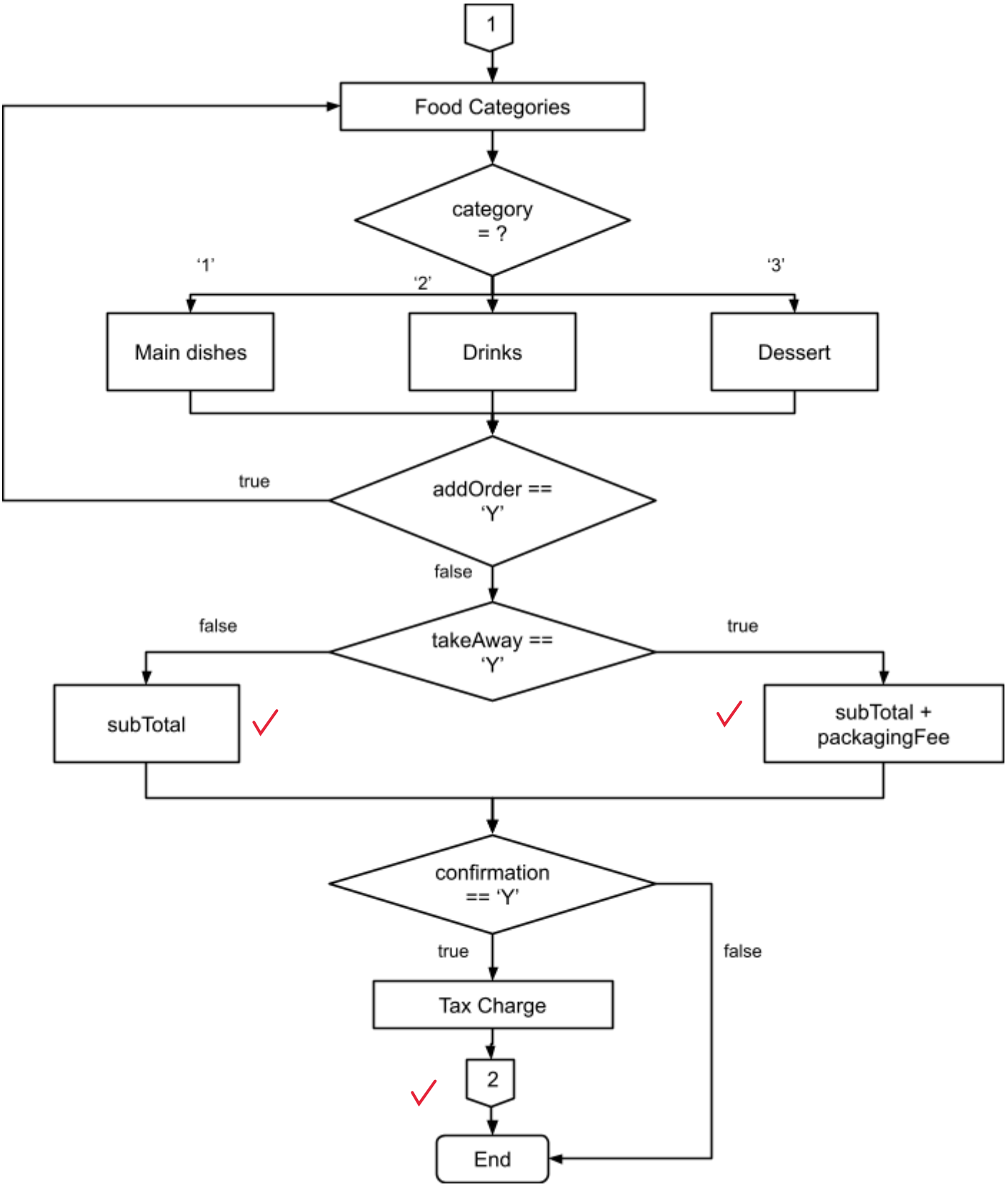
Item	Quantity	Price (RM)	Total (RM)
… e.g. Pasta	… 2	… 12.90	REVENUE = SLSRPQTY * FOOD_PRICE 2 * 12.90 = 25.80
: :	: :	: :	: :
Total Revenue			$\sum_{i=1}^{n}$ REVENUE

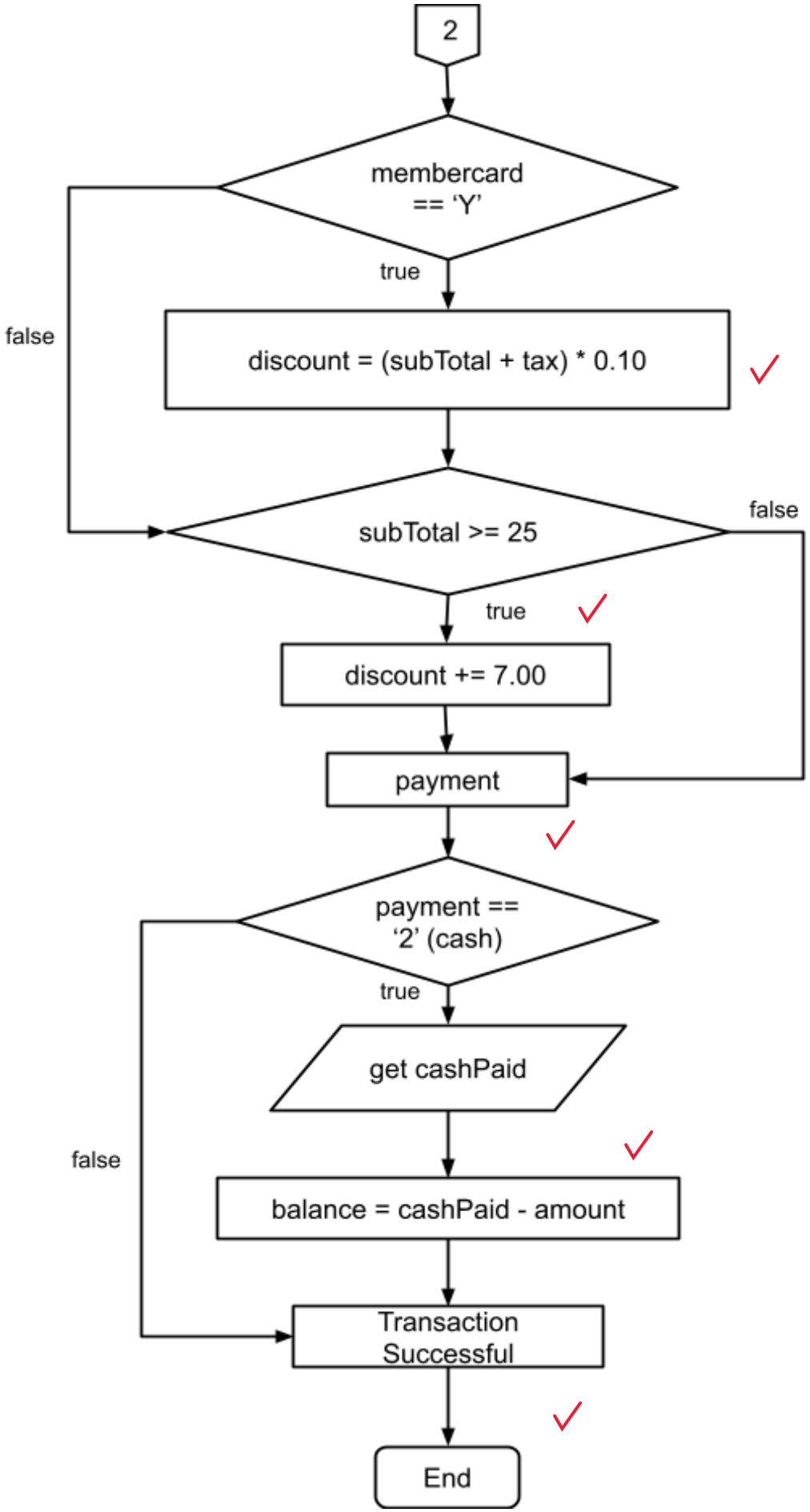
- **Stock Tracking**
  Every items are prepared by default of 50 sets
  qtySold = 50 - stockLeft
  qtyLeft = stockLeft

  Code variable:
  Quantity sold - AL = 50 - FOOD_STOCK[SI]
  Quantity left - AL = FOOD_STOCK[SI]

Flowchart


Start
Login
false
true
ID and password validation
Main menu
selection = ?
'1'
'2'
'3'
'4'
'5'
'6'
1
Stock tracking
Administrator access validation
Administrator access validation
Log out
Quit system
false
true
false
true
Sales report
Register
End


1
Food Categories
category
= ?
'1'
'2'
'3'
Main dishes
Drinks
Dessert
true
addOrder ==
'Y'
false
false
takeAway ==
'Y'
true
subTotal
subTotal +
packagingFee
confirmation
== 'Y'
true
false
Tax Charge
2
End


2
membercard == 'Y'
true
false
discount = (subTotal + tax) * 0.10
subTotal >= 25
false
true
discount += 7.00
payment
payment == '2' (cash)
true
get cashPaid
false
balance = cashPaid - amount
Transaction Successful
End

User Manual with Screenshots

Screenshot	Description
Login	
Wrong ID: Wrong password: 	Initial login (before registering new user) user: ***admin*** password***: password*** When the user enters the wrong user ID or password, the system prompts again until the user enters the correct ID and password. Password is also masked (*) to reduce the risk of password leakage.

Correct ID and password:

```
Login Successful.

==========================================
|                Welcome                 |
|  ____        _ _        _____ _       |
| |  _ \  __ _(_)|_    _  |  ___(___  __ |
| | | | |/ _` | | | | | | |_  | \ \/ / |
| | |_| | (_| | | | |_| | |  _| | |>  < |
| |____/ \__,_|_|_|\__, | |_|   |_/_/\_\ |
|                  |___/                 |
|               POS system               |
==========================================

------------------------------------------
|                MAIN MENU               |
------------------------------------------

     1. Take Order
     2. Stock Tracking
     3. Sales Report
     4. Register User ID
     5. Log Out
     6. Quit System

     Enter your choice: _
```

Once the user enters the right password, "Login Successful" and the Daily Fix POS system menu will be displayed. The user can enter number (1-6) to proceed to the desired page according to the menu.

Main Menu

Example input: *?*

```
|                MAIN MENU               |
------------------------------------------

     1. Take Order
     2. Stock Tracking
     3. Sales Report
     4. Register User ID
     5. Log Out
     6. Quit System

     Enter your choice: ?
     ! ! Invalid input. Only range (1 - 6) accepted.
```

Example input: *7*

```
|                MAIN MENU               |
------------------------------------------

     1. Take Order
     2. Stock Tracking
     3. Sales Report
     4. Register User ID
     5. Log Out
     6. Quit System

     Enter your choice: 7
     ! ! Invalid input. Only range (1 - 6) accepted.
```

Validation of user input is set in the main menu section where only "1 – 6" is accepted. Digits below 1, above 6 and other characters will prompt an error message and require the user to input again.

By entering each valid input, it brings the user to different functions designed in our system. ✓

✓

Take Order

```
Enter your choice: 1

----------------------------------------
|                ORDER                 |
----------------------------------------

1. Main Dishes
2. Dessert
3. Drinks

Enter your choice: _
```

When the user enters **"1"** in the main menu, it brings the user to the ordering page.
The category of food will be displayed for the user to choose from.

```
----------------------------------------
|                ORDER                 |
----------------------------------------

1. Main Dishes
2. Dessert
3. Drinks

Enter your choice: 5
! ! Invalid input. Only range (1 - 3) accepted.

----------------------------------------
|                ORDER                 |
----------------------------------------

1. Main Dishes
2. Dessert
3. Drinks

Enter your choice:
```

If the user enters a number that is out of range (1-3), the system will prompt an error message and display the order category again for the user to choose.

Multi-Level Menu (Different Menu Category)

Main dishes menu:

```
Enter your choice: 1

========================================
|  __  __      _        ___  _    _    |
| |  \/  |__ _(_)_ _   |   \(_)__| |_  |
| | |\/| / _` | | ' \  | |) | (_-| ' \ |
| |_|  |_\__,_|_|_||_| |___/|_/__|_||_| |
|                                      |
|                MENU                  |
========================================

  ------------------------------------
  |No. |Food                |Price(RM)|
  ------------------------------------
  | 1. | Chicken Pasta      | 10.00   |
  | 2. | Tomato Soup        | 7.00    |
  | 3. | Curry Rice         | 12.00   |
  | 4. | Creamy Carbonara   | 12.00   |
  | 5. | Beef Steak         | 13.00   |
  ------------------------------------

Enter option:
```

When the user enters a number that is valid (1-3), the system will display the menu according to the category chosen by the user. For example, when the user enters **"1"**, the menu of the main dish will be displayed as in the screenshot.

Dessert menu:

```
================================
|  ___                     _    |
| |   \ ___ ________ _ _| |_   |
| | |) / -_(_-(_-/ -_| '_|  _| |
| |___/\___/__/__\___|_|  \__| |
|                              |
|            MENU              |
================================

  ------------------------------------
  |No. |Food                |Price(RM)|
  ------------------------------------
  | 1. | Cheese Cake        | 8.00    |
  | 2. | Pudding            | 5.00    |
  | 3. | Ice Cream          | 7.00    |
  ------------------------------------

     Enter option: _
```

When the user enters **"2"**, the menu of the dessert will be displayed as in the screenshot.

Drinks menu:

```
================================
|                              |
|  ___  ___ ___ _  _ _  _____  |
| |   \| _ |_ _| \| | |/ / __| |
| | |) |   /| || .` | ' <\__ \ |
| |___/|_|_|___|_|\_|_|\_|___/ |
|                              |
|            MENU              |
================================

  ------------------------------------
  |No. |Food                |Price(RM)|
  ------------------------------------
  | 1. | Smoothies          | 8.00    |
  | 2. | Iced Latte         | 9.00    |
  | 3. | Hot Chocolate      | 9.00    |
  | 4. | Lemon Tea          | 8.00    |
  ------------------------------------
     Enter option:
```

When the user enters **"3"**, the menu of the drinks will be displayed as in the screenshot.

Food options and Quantity

```
==========================================
|  __  __      _        ___  _    _      |
| |  \/  |__ _(_)_ _   |   \(_)__| |_    |
| | |\/| / _` | | ' \  | |) | (_-| ' \   |
| |_|  |_\__,_|_|_||_| |___/|_/__|_||_| |
|                                          |
|                  MENU                    |
==========================================

  ------------------------------------
  |No. |Food                 |Price(RM)|
  ------------------------------------
  | 1. | Chicken Pasta       | 10.00   |
  | 2. | Tomato Soup         | 7.00    |
  | 3. | Curry Rice          | 12.00   |
  | 4. | Creamy Carbonara    | 12.00   |
  | 5. | Beef Steak          | 13.00   |
  ------------------------------------

     Enter option:
```

Take an example of Main Dish menu:
The user is to enter the number of options (1-5) desired and the quantity wanted.

Example input: *7*

```
Enter option: 7
! ! Invalid input. Only option (1 - 5) accepted.

Enter option:
```

An error message will be displayed if the user enters the option which is out of range and prompt for the user to enter the option again.

Example input: *3*

```
Enter option: 3
Enter quantity (1 - 9):
```

It proceeds to enter the quantity of food to order after a valid input.

Example wrong input: ***0*** **,** +
Example correct input: ***2***

```
Enter quantity (1 - 9): 0
! ! Invalid input. Only quantity (1 - 9) accepted.
Enter quantity (1 - 9): +
! ! Invalid input. Only quantity (1 - 9) accepted.
Enter quantity (1 - 9): 2
Add order? (Y/N):
```

Digits below 1, above 9 and other characters will prompt an error message and require the user to input again.

Adding order

Example input: ***Y*** (Yes)

```
Add order? (Y/N): Y

------------------------------------------
|                ORDER                   |
------------------------------------------

    1. Main Dishes
    2. Dessert
    3. Drinks

    Enter your choice: _
```

After a valid input is entered, it will ask the user whether to add any other order again.

If yes **"Y"/"y"**, the category of food will be displayed for the user to make an additional order.

Example input: ***N*** (No)

```
Add order? (Y/N): N

Take Away ? (Y/N):
```

If the user enters **"N"/"n"**, it means the user does not want to make any additional orders.
User will be asked whether he/she wants the order(s) to be taken away.

```
Add order? (Y/N): j
! ! Invalid input. Only 'Y' / 'N'.
Add order? (Y/N): :
! ! Invalid input. Only 'Y' / 'N'.
Add order? (Y/N): _
```

An error message will be prompted if user enters an invalid choice other than (Y / y / N/ n).

Display Order Summary

Example input: ***Y*** (Yes)

```
Take Away ? (Y/N): Y

-----------------------------------------
|                PAYMENT                |
-----------------------------------------

-----------------------------------------------------
 Food                Qty  Price(RM)    Subtotal(RM)
-----------------------------------------------------
Chicken Pasta        02     10.00          20.00
Beef Steak           01     13.00          13.00
Cheese Cake          01     08.00          08.00
Pudding              01     05.00          05.00
-----------------------------------------------------
 Packaging Fee                             0.20
-----------------------------------------------------
=====================================================
 Grand Total                              46.20
=====================================================
Confirm (Y/N): _
```

If yes **"Y"/"y"**, to take away, an addition of RM0.20 packaging fee will be added to the grand total.

Example input: ***N*** (No)

```
Take Away ? (Y/N): N

-----------------------------------------
|                PAYMENT                |
-----------------------------------------

-----------------------------------------------------
 Food                Qty  Price(RM)    Subtotal(RM)
-----------------------------------------------------
Chicken Pasta        02     10.00          20.00
Beef Steak           01     13.00          13.00
Cheese Cake          01     08.00          08.00
Pudding              01     05.00          05.00
=====================================================
 Grand Total                              46.00
=====================================================
Confirm (Y/N):
```

If **"N"/"n"**, no take away, no packaging fee added.

Calculation:

Order price = (2 x 10.00) + (1 x 13.00) + (1 x 8.00) + (1 x 5.00)
= 20.00 + 13.00 + 8.00 + 5.00
= 46.00

<u>Case 1 (take away)</u>

Grand Total = 46.00 + 0.20
= 46.20

<u>Case 2 (no take away)</u>

Grand Total = 46.00 + 0.00
= 46.00

Order Confirmation and Tax Charge

Example input: ***Y*** (Yes - confirm order)

```
Confirm (Y/N): Y

-----------------------------------------
|              TAX CHARGE               |
-----------------------------------------

Subtotal: 46.20

-----------------------------------------------------
Service Tax (SST 6%): 2.77
Service charge (10%): 4.62
-----------------------------------------------------
Total (added tax): 53.59
-----------------------------------------------------


Membercard (Y/N): _
```

```
Confirm (Y/N): Y

-----------------------------------------
|              TAX CHARGE               |
-----------------------------------------

Subtotal: 46.00

-----------------------------------------------------
Service Tax (SST 6%): 2.76
Service charge (10%): 4.60
-----------------------------------------------------
Total (added tax): 53.36
-----------------------------------------------------


Membercard (Y/N):
```

If the user has confirmed to place an order (**"Y"/"y"**), the subtotal, SST tax, service charge and total including tax will be displayed. User is asked whether the customer has a membership.

Calculation:

<u>Case 1</u>

SST = 46.20 x 0.06 = 2.77
Service charge = 46.20 x 0.10 = 4.62
Total including tax = 46.20 + 2.77 + 4.62
= 53.59 ✓

<u>Case 2</u>

SST = 46.20 x 0.06 = 2.76
Service charge = 46.20 x 0.10 = 4.60
Total including tax = 46.20 + 2.77 + 4.62
= 53.36

✓

Example input: ***n*** (No - cancel order)

```
Confirm (Y/N): n

=========================================
|              Welcome                  |
|  ____       _ _        _____ _       |
| |  _ \  __ _(_)_|_    _  |___(___  __ |
| | | | |/ _` | | | | | | |_  | \ \/ / |
| | |_| | (_| | | | |_| | |  _| | |>  < |
| |____/ \__,_|_|_|\__, | |_|   |_/_/\_\ |
|                   |___/               |
|              POS system               |
=========================================

-----------------------------------------
|              MAIN MENU                |
-----------------------------------------

     1. Take Order
     2. Stock Tracking
     3. Sales Report
     4. Register User ID
     5. Log Out
     6. Quit System

     Enter your choice: _
```

If "N"/"n", does not confirm the order, the system will loop back to the main menu.

Discount Voucher

Example input: ***Y*** (Yes)

```
Membercard (Y/N): Y

----------------------------------------
|            Discount Voucher           |
----------------------------------------
| Member Discount - 10%                 |
| SAVE7 Voucher for purchase over RM 25 |
----------------------------------------

Discount: 12.35

Total Payment: 41.24

          Payment Option:
          1. Card
          2. Cash
          3. E-Wallet

     Enter option:
```

Example input: ***N*** (No)

```
Membercard (Y/N): N

----------------------------------------
|            Discount Voucher           |
----------------------------------------
| Member Discount - 10%                 |
| SAVE7 Voucher for purchase over RM 25 |
----------------------------------------

Discount: 7.00

Total Payment: 46.36

          Payment Option:
          1. Card
          2. Cash
          3. E-Wallet

     Enter option: _
```

RM7 will be deducted as a discount if the order is above RM25. If the user has a membership an additional 10% will be discounted. Total payment will be displayed.

Calculation:

<u>Case 1 (Member)</u>
Subtotal: 53.59
Member = 53.59 x 0.10 = 5.35
SAVE7 (>25) = 7.00
Discount = 5.35 + 7.00 = 12.35
Total payment = 53.59 - 12.35 = 41.24

<u>Case 2 (No member)</u>
Subtotal: 53.36
Member = 0.00
SAVE7 (>25) = 7.00
Discount = 0.00 + 7.00 = 7.00
Total payment = 53.36 - 7.00 = 46.36

Payment Method and Balance Calculation

```
          Payment Option:
          1. Card
          2. Cash
          3. E-Wallet

     Enter option: _
```

The user will be able to choose their preferred payment method among card, cash, or e-wallet.

Example invalid input: ***4***

```
      Payment Option:
      1. Card
      2. Cash
      3. E-Wallet

  Enter option: 4
  ! ! Invalid input. Only option (1 - 3) accepted.

  Enter option:
```

Validation of payment option range (1 – 3) only, error message prompt and request to input again.

Example input: ***3*** (E-wallet)

```
----------------------------------------
¦        Transaction Processing        ¦
----------------------------------------

          Transaction Successful !


       Press any key to continue ... _
```

If the user chooses card (1) or e-wallet (3), it will display transaction status only.

Example input: ***2*** (cash)
Cash paid input: ***050.30***

```
Total Payment: 41.24

        Payment Option:
        1. Card
        2. Cash
        3. E-Wallet

    Enter option: 2

    ----------------------------------------
    ¦        Transaction Processing        ¦
    ----------------------------------------

    Enter amount paid (max limit 300.00): 050.30

    Balance: 9.06

            Transaction Successful !


        Press any key to continue ... _
```

The balance will be shown if the customer is paying by cash.

Calculation:
<u>Case 1</u>
Total payment: 41.24
Cash paid: 50.30
Balance = 50.30 - 41.24 = 9.06

Example error input for cash paid: ***320.00***

```
----------------------------------------
¦        Transaction Processing        ¦
----------------------------------------

Enter amount paid (max limit 300.00): 320.00

! Invalid input, cash range (000.00 -- 300.00) only.
```

Validation for cash paid in two cases:

- Invalid input or out of range will prompt an error message and ask to enter again.
- Cash entered is less than the total payment required will tell the user and ask to enter again.

Example cash paid insufficient: ***010.00***

```
----------------------------------------
¦        Transaction Processing        ¦
----------------------------------------

Enter amount paid (max limit 300.00): 010.00

! Cash not enough. Please reenter.
```

Upon a successful payment, the system will loop back to the main menu.

Stock Tracking

```
Items Sold            Qty Sold     Qty Left
Chicken Pasta          2            48
Tomato Soup            0            50
Curry Rice             0            50
Creamy Carbonara       0            50
Beef Steak             1            49
Cheese Cake            1            49
Pudding                1            49
Ice Cream              0            50
Smoothies              0            50
Iced Latte             0            50
Hot Chocolate          0            50
Lemon Tea              0            50

Press any key to continue ... _
```

The default stock available is 50 for every food item. The stock amount will be deducted according to the quantity of food being ordered previously:

2 x Chicken pasta
1 x Beef steak
1 x Cheese cake
1 x Pudding

```
Enter option: 2
Enter quantity (1 - 9): 9
Sorry, not enough stock, please reenter. Stock remain: 5
```

Additionally, in the order function, a message will be prompted and the amount left will be displayed when the amount of quantity for the chosen food is insufficient.

```
Items Sold            Qty Sold     Qty Left
Chicken Pasta          3            47
Tomato Soup           47             3      !! Lack of stock
Curry Rice             0            50
```

A message "**!! Lack of stock**" will be displayed at the side of the food item running out of stock when the remaining stock is less than or equal to 5.

Administrator Validation

Case correct admin password:

```
Enter your choice: 3

Administrator Validation

User ID: admin
Password: *******_
```

Case incorrect admin password:

```
Administrator Validation

User ID: admin
Password: ********
Incorrect password, cannot proceed. Please try again.
```

Administrator validation is required to proceed with viewing the sales report and registering new users.

Sales Report

```
Person in Charge: admin                Date(DD-MM-YY): 17/04/24

Total Customers: 1
-----------------------------------------------------------------------

                Press any key to continue ... _
```

The person in charge (the one logging in the system currently), date, and total customers will be displayed after the administrator validation.

```
-----------------------------------------------------------------------
 Item                          Qty Sold     Price(RM)        Total
-----------------------------------------------------------------------
 Chicken Pasta                    2           10.00           20.00
 Tomato Soup                      0            7.00           00.00
 Curry Rice                       0           12.00           00.00
 Creamy Carbonara                 0           12.00           00.00
 Beef Steak                       1           13.00           13.00
 Cheese Cake                      1            8.00           08.00
 Pudding                          1            5.00           05.00
 Ice Cream                        0            7.00           00.00
 Smoothies                        0            8.00           00.00
 Iced Latte                       0            9.00           00.00
 Hot Chocolate                    0            9.00           00.00
 Lemon Tea                        0            8.00           00.00
-----------------------------------------------------------------------
 Total Revenue:                                               046.00
-----------------------------------------------------------------------
 Best Seller: Chicken Pasta

                Press any key to continue ... _
```

The items sold, quantity sold, price per item and total for each item sold are displayed clearly.
Total revenue will also be calculated and displayed at the bottom.

Register User ID

```
Enter role [S]erver / [C]lashier: S

User ID: user1
Enter password (8 characters): psw12345
Confirm password: psw12333
!! Password not validated, please try again.

User ID: user1
Enter password (8 characters): psw12345
Confirm password: psw12345

     Register successful!

     Press any key to continue ...
```

User ID will be assigned automatically, and users can key in their password (8 characters). The password needs to be keyed in twice for confirmation.

The entered confirmed password is validated before a new user account can be registered. Error message is prompt and required to reenter.

Invalid role input entered:

```
Enter role [S]erver / [C]lashier: H
!! Invalid input. Only 'S' or 'C' accepted.

Enter role [S]erver / [C]lashier: c
```

Role validation where only “S” / “s” / “C” / “c” is accepted. Error message is prompt and required to reenter.

```
Enter your choice: 4

Sorry, maximum number of users reached.

Press any key to continue ... _
```

Note that only three new users can be registered. A short message will tell the user about the maximum number of users reached after the limit reached.

Log Out

```
Enter your choice: 5

==========================================
|                Welcome                 |
|  ____        _ _          _____ _      |
| |  _ \  __ _(_)|_    _  |  ___(___  __ |
| | | | |/ _` | | | | | | |_  | \ \/ / |
| | |_| | (_| | | | |_| | |  _| | |>  <  |
| |____/ \__,_|_|_|\__, | |_|   |_/_/\_\ |
|                  |___/                 |
|                POS system              |
==========================================

------------------------------------------
|                LOGIN                   |
------------------------------------------

Enter ID:
```

When the user enters option **"5"** from the main menu which indicates log out, the user will be logged out and directed back to the login page.

```
==========================================
|                Welcome                 |
|  ____        _ _          _____ _      |
| |  _ \  __ _(_)|_    _  |  ___(___  __ |
| | | | |/ _` | | | | | | |_  | \ \/ / |
| | |_| | (_| | | | |_| | |  _| | |>  <  |
| |____/ \__,_|_|_|\__, | |_|   |_/_/\_\ |
|                  |___/                 |
|                POS system              |
==========================================

------------------------------------------
|                LOGIN                   |
------------------------------------------

Enter ID: user2
Enter Password: *******
```

Users can now login with the registered account.

Quit System

```
Enter your choice: 6

==================================================
|                System Close                    |
|  _____ _                 _    __    __        |
| |_   _| |_  __ _ _ _ _ | |__ \ \ / ___ _  _   |
|   | | | ' \/ _` | ' \| / / / \ V / _ | || | |
|   |_| |_||_\__,_|_||_|_\_\   |_|\___/\_,_| |
|                                                |
| for using Daily Fix Restaurant POS System      |
|               Have a nice day!                 |
==================================================
```

Thank you message is prompted when the user quits the Daily Fix Restaurant POS system with option **"6"** from the main menu.

Advanced Features

1. **Capture System Date**
   To get system date [DD/MM/YY] , store day into DL, month into DH and year into EX.
   Add 0FE30h into CX to adjust hexadecimal effects on year, store CX into AX.
   Call SHOW procedure to display day, month and year.

   SHOW procedure:
   Using *AAM* for ASCII adjustment after multiplication (*AX* register). Store *AX* into *BX* to load adjusted value to *BX*, add 3030h into *BX* to print the data properly.
   Store *BH* into *DL* to print the first digit of data, store *BL* into *DL* to print the second digit.
   Return to the *SHOW* procedure and end display.

2. **Procedure Calling**
   Procedure calling is an additional feature not covered in our practical syllabus. It is a systematic method and is part of the code that can be called to a place in order to perform a specific task. It helps to reduce the repeating part of code by creating reusable code that can be executed upon calling. This enhances the dynamic use of our program code and makes our program more structured and easy to understand. After being executed, it will return to the point where it is being called.

   It can be defined with syntax:
   ```
   name PROC
     (coding)
   RET
   name ENDP
   ```
   Call with syntax: *CALL name*

3. **Applying Color to Text**
   Certain sentences of output are being applied with color. The error message prompt will be in red color. Other status or important messages will be displayed in cyan or yellow color to attract user attention and alert the user.
   Color code defined:
   ```
   RED DB 0CH
   YELLOW DB 0EH
   CYAN DB 0BH
   ```

   Syntax used to apply color in our system:
   ```
   MOV BH,0
   MOV BL,colorCode
   MOV CX,columnLength
   INT 10H
   ```

4. **New User Registration**
   The system allows the user to register a new user account for login. The user ID is automatically generated for the user. However, it needs to be able to save the password created by the user and identify the password for different user accounts respectively in an efficient and appropriate manner.

   A variable *NUMUSERS* is used to identify the number of users currently available for logging in the POS system. It also helps to indicate the position of character in an array should be stored to or retrieved from. We used *USERID DB "admin$","user1$","user2$","user3$"* to store user ID and check for the respective password that stored in *USERPSW DB "password$", 9 DUP ('$'), 9 DUP ('$'), 9 DUP ('$').*

   Besides, we also set numeric constants for user ID length, *ID_LENGTH DB 6* and password length, *PSW_LENGTH DB 9* for array handling. For example when the second user (user1) with index 1 needs to be accessed, the index will multiply by the *ID_LENGTH* or *PSW _LENGTH*, so the starting character to be printed out and validate password is 1 x 6 = 6th index (7th character) for user ID and 1 x 9 = 9th index (10th character).

5. **Storing String in Array**
   Similar to the method used for user ID and password, we store the food item name (strings) in an array:
   FOOD_NAME DB "Chicken Pasta $","Tomato Soup $","Curry Rice $","Creamy Carbonara$"
   DB "Beef Steak $","Cheese Cake $","Pudding $","Ice Cream $"
   DB "Smoothies $","Iced Latte $","Hot Chocolate $","Lemon Tea $"
   In this way, we can easily print the required output during the order summary, sales report and stock quantity in an efficient manner.
   MOV AH,09H
   LEA DX,FOOD_NAME[BX]
   INT 21H

6. **Allowed to Cancel Order**
   After displaying the order summary, if the user does not confirm the order, the order will be canceled and loop back to the menu. However, there is a consideration that should be aware of, which is the items currently chosen by the user, stored in *USER_CHS[SI]* should be added back to the food stock, *FOOD_STOCK[SI]* using looping and array handling methods.

7. **Complex Calculation**
   a) **Correct handling of decimal value**
   Decimal values are common in POS systems, especially when dealing with prices and taxes.
   We use a series of variables: *Q1, R1, Q2, R2, Q3, R3* to store the results digit by digit.

   Example:
   The results to be displayed such as *SUBTOTAL* of 13520 (RM 135.20) is moved into the AX register. Then, it is divided by *HDR (DB 100)* to separate the integer part and decimal point into *Q1* and *CENTS* respectively. After that, the digits are being extracted one by one when dividing them by *TEN (DB 10)*.

   13520 / 100 = 135.20 (AL:AH)
   Q1 = 135 CENTS = 20
   135 / 10 = 13.5
   Q1 = 13 R1 = 5
   13 / 10 = 1.3
   Q2 = 1 R2 = 3 ✓

   Add *Q2, R2, R1* with 30H and display them followed by a dot ".".
   The cents goes with the same way:

   CENTS = 20
   20 / 10 = 2.0
   Q1 = 2 R1 = 0

   Output: 135.20

   b) **Order Subtotal Summary**
   We use looping to check for the ordered items. If the value stored in *USER_CHS[SI]* is not 0, then the food, price and its total is being printed.

   Example: 2 x Cheese cake (RM 8.00) ✓

   Put quantity into *AL* register: AL = USER_CHS[SI] (2)
   Put food price into *BL* register: BL = FOOD_PRICE[SI] (8)
   Multiply them: MUL BL → AL = 2 x 8 = 16
   Add the result into *SUBTOTAL*: SUBTOTAL += AX ✓

c) Tax Charge

- **SST (Government Tax 6%)**
  Example: SUBTOTAL = 4600 (RM 46.00)

  Put subtotal into *AX* register: AX = 4600
  AX = 4600 / HDR (DB 100) * SIX (DB 6)
  = 46 * 6
  = 276
  Put *AX* into *SSTTAX*: SSTTAX = 276 (RM 2.76)

- **Service Charge (10%)**
  Put subtotal into *AX* register: AX = 4600
  AX = 4600 / HDR (DB 100) * TEN (DB 10)
  = 46 * 10
  = 460
  Put *AX* into *SERVICETAX*: SERVICETAX= 460 (RM 4.60)

Total After Tax
SUBTOTAL += SSTTAX + SERVICE TAX

d) Discount / Voucher

- **Member Discount (10%)**
  Condition: If input for member = "Y"/"y"
  CMP AL,'Y'
  JE MEMBERDISC
  CMP AL,'y'
  JE MEMBERDISC

  Example: SUBTOTAL = 5336 (RM 53.36)

  Put subtotal into *AX* register: AX = 5336
  AX = 5336 / HDR (DB 100)
  = 53.36
  Store into *Q1* and *CENTS*: Q1 = 53 CENTS = 36

  Divide *CENTS* with 10:
  36 / 10 = 3.6
  Only take value of the quotient:
  MOV Q2,AL
  Q2 = 3

  Put *Q1* into *AL*: AX = 53
  Multiply 10: AX = 53 * 10 = 530
  Add *Q2*: AX = 530 + 3 = 533

  Put result into DISCOUNT: DISCOUNT = 533

- **RM 7 Voucher (SAVE7)**
  Condition: If subtotal > 25
  CMP SUBTOTAL,2500

  DISCOUNT += 700 (RM 7.00)

Total Payment
SUBTOTAL – = DISCOUNT

e) Balance

The cash payment entered by the user is stored in the variable *INPUT1* through *INPUT5*.
After validation of input digit by digit (within range 300.00), the input digits of are being converted to corresponding cash paid one by one:

INPUT1 * TENTSD (DW 10000)
INPUT2 * THOUSAND (DW 1000)
INPUT3 * HUNDRED (DW 100)
INPUT4 * TEN (DB 10)
INPUT5

Add them all into *CASHPAID*.

Put *CASHPAID* into *AX* and minus the *SUBTOTAL*:

AX = CASHPAID - SUBTOTAL

Then put the result into *BALANCE*:

BALANCE = AX

If the *BALANCE* is less than 0, it will jump to insufficient error message.

Conclusion and Future Improvements

Conclusion:

The restaurant's operations will benefit greatly from the implementation of the Daily Fix Restaurant Point of Sales (POS) system, which will speed up procedures, increase productivity, and raise customer satisfaction levels overall. The POS system lessens staff workload, lowers the possibility of human mistakes, and empowers restaurant management to make well-informed decisions by automating processes like order processing, payment calculation, and inventory management. The system's strong security measures protect confidential information, and its vast reporting features offer insightful data on consumer preferences and sales performance.

Future improvements:

1) **Automated formatting (flexibility of input for cash payment less than 3 digits)**
   Change the POS system to automatically format cash payments without requiring the leading zero(s). For instance, if a customer pays RM50 for the order placed, the user doesn't need to add the leading zero (050.00) when entering. Instead, they can directly enter "50".

2) **Handling larger value**
   Use 32-bit or 64-bit registers for storing and handling higher payment amounts for greater precision and overflow protection. Upgrade the POS system's assembly language to make use of additional registers in order to hold greater values for cash payments. Provide more registers to support values greater than the 255 16-bit limit. Adjust the assembly code's calculations to accommodate computations with larger values located in several registers.

3) **Transaction history**
   Modify the point-of-sale system to record transaction information for every order. Create a data structure to hold transactional data, which should include the date, time, items ordered, payment method, total amount, and any other relevant data. Modify the code to capture related transaction information and store it in a suitable data structure.

4) **User ID input**
   Apply changes to the registration procedure so that users are prompted to enter their password and desired ID when registering. Set validation procedures to make sure the user-provided ID fulfils all necessary requirements, such as length, format, and uniqueness. Verify that the ID provided by the user doesn't exist yet in the system to avoid ID duplication. Prompt the user to enter a new ID if the ID is not unique.